FECAP – CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO

Curso de Ciência da Computação
Disciplina: Sistemas Operacionais e Computação em Nuvem
Professor: Rodrigo da Rosa

Saulo Ribeiro Santos
Gustavo Henrique Da Silva Santos
Luan Rocha da Silva
Thiffany Morais Vieira da Silva

ASA CONNECT — AGENTE PARA O ESTUDANTE

Relatório Técnico de Infraestrutura — Entrega 2

Deploy, Observabilidade, Segurança e Desempenho

São Paulo
2026

1 OBJETIVO

Este relatório cumpre a Entrega 2 da disciplina de Sistemas Operacionais e Computação em Nuvem. O objeto é o ASA Connect+ (Agente para o Estudante), da Área do Sucesso Alvarista da FECAP: a API FastAPI, o banco e o agente que estão em src/backend. A equipe mudou o rumo da infraestrutura e documenta essa solução, e não um ambiente Linux genérico. O texto cobre o empacotamento Docker, as variáveis de ambiente, a conexão com o banco, a subida da API e do agente, os logs, o consumo de CPU e memória, o gargalo observado, o controle de acesso, a gestão de segredos e a estratégia de backup. O endereço controlado da execução é http://localhost:8080.

O relatório cobre: (i) o empacotamento da API e das dependências em Docker, com variáveis de ambiente e conexão ao banco; (ii) a subida da API, do banco e do agente, além dos descritores de nuvem que o repositório já contém; (iii) logs estruturados e a leitura de CPU, memória, latência e erros; (iv) o gargalo observado na inicialização da base e o ajuste indicado; e (v) backup e restauração, gestão de segredos, controle de acesso e estratégia de recuperação.

2 DESENVOLVIMENTO

2.1 Empacotamento da API e das dependências

A API do ASA Connect+ está em src/backend e é empacotada pelo Dockerfile dessa pasta. A imagem usa python:3.12-slim em um único estágio. O Dockerfile instala build-essential, libpq-dev e curl, instala o requirements.txt e cria o usuário não-root user, com UID 1000. A instrução USER user encerra o arquivo. A porta padrão da imagem é 7860. Na execução, a variável PORT foi definida como 8080, e o processo foi uvicorn app.main:app --host 0.0.0.0 --port 8080. A imagem resultante se chama asaconnect-api:2.0.0. Não há segundo estágio: os pacotes de compilação permanecem na imagem final. O Dockerfile não declara HEALTHCHECK nem sobe o Uvicorn com mais de um worker.

Comandos utilizados para construir a imagem e conferir o usuário de execução, a partir de src/backend:

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
docker exec asaconnect_api whoami
```

O comando de subida constrói a imagem asaconnect-api:2.0.0, cria a rede backend_default e o volume backend_postgres_data, e inicia os contêineres asaconnect_postgres e asaconnect_api. O comando whoami consulta a identidade do processo dentro da API. Na execução de 8 de novembro de 2026, a resposta foi user.

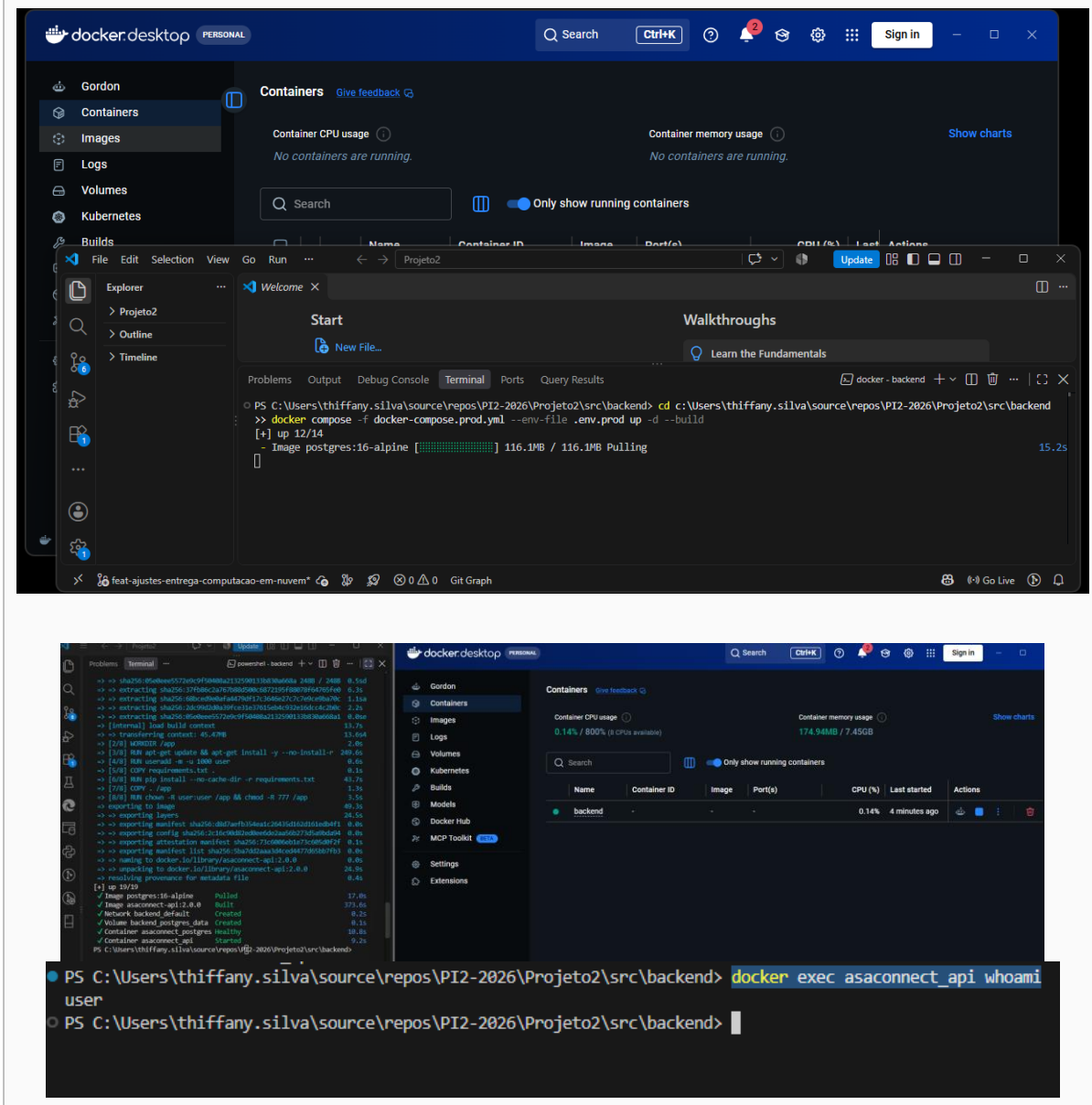


Figura 1 – Build da imagem asaconnect-api:2.0.0 e identidade user do contêiner da API.
Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: Os comandos foram executados para comprovar que a API sobe a partir do Dockerfile do projeto e que o processo atende com o usuário user, sem privilégio de root.

2.2 Variáveis de ambiente e conexão com o banco

O aplicativo Flutter não se conecta ao PostgreSQL. Ele chama a API por HTTP. Quem abre o banco é a API, por meio da variável DATABASE_URL, lida em app/core/config.py. O padrão do código e do arquivo .env.example é o SQLite asaconnect.db. Na execução evidenciada, a DATABASE_URL apontou para o

PostgreSQL 16 do serviço postgres. O fly.toml e o render.yaml do repositório ainda declaram SQLite.

A senha não está no código nem neste relatório. O arquivo .env.prod, fora do Git, guarda POSTGRES_PASSWORD, JWT_SECRET_KEY e GEMINI_API_KEY. O modelo versionado é .env.example. O .gitignore bloqueia .env e .env.*, e libera .env.example. O Compose monta a conexão com o usuário asaconnect, o banco asaconnect_db, o host interno postgres e a porta 5432. A forma da URL, sem o valor da senha, é postgresql://asaconnect:<POSTGRES_PASSWORD>@postgres:5432/asaconnect_db. Dentro da rede Docker o host é o nome do serviço. No docker ps, o PostgreSQL aparece apenas como 5432/tcp, sem porta publicada no computador.

Trecho de referência do modelo .env.example, sem valores reais:

```
POSTGRES_USER=asaconnect
POSTGRES_PASSWORD=<definida no .env.prod>
POSTGRES_DB=asaconnect_db
JWT_SECRET_KEY=<definida no .env.prod>
GEMINI_API_KEY=
```

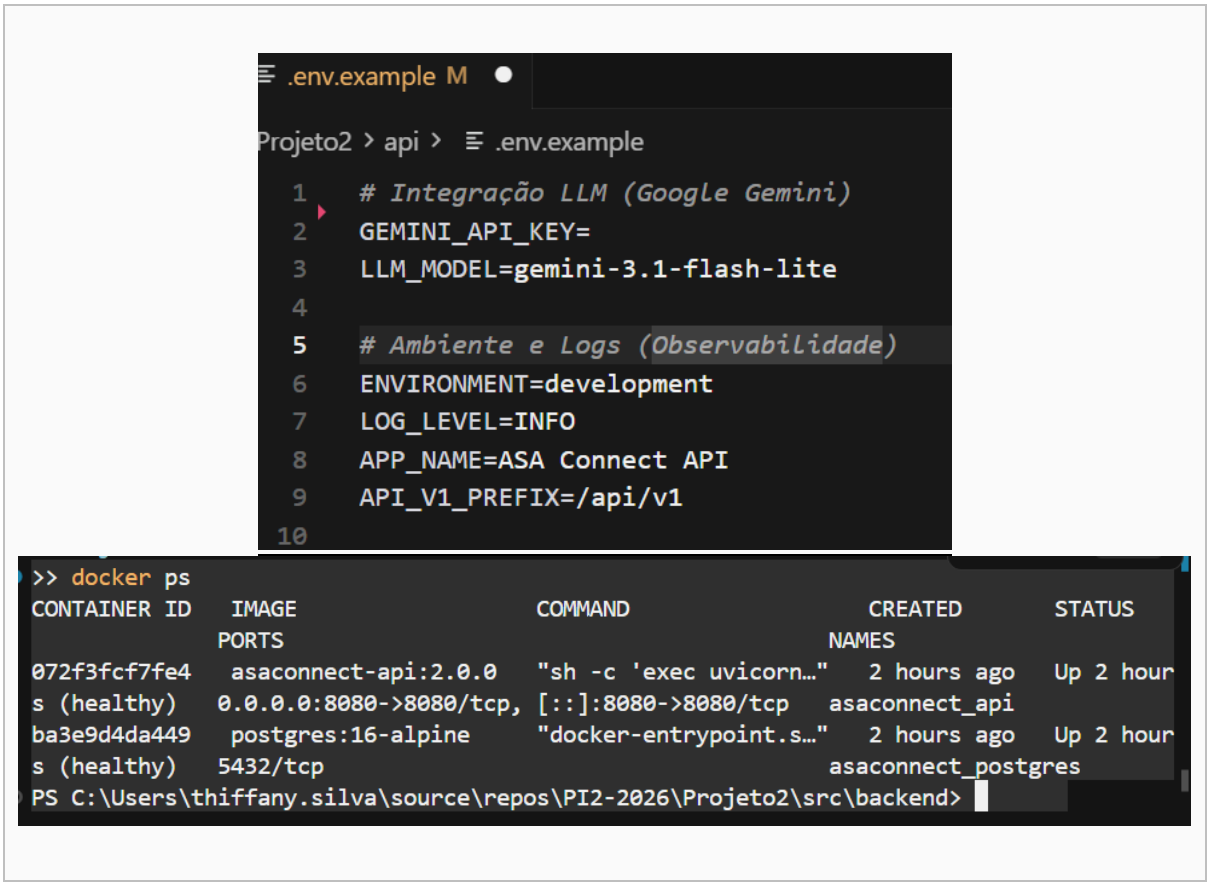


Figura 2 – Modelo de variáveis de ambiente, sem segredos, e ausência da senha no código versionado.

Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: A verificação mostra que a conexão com o banco e as chaves ficam em variável de ambiente, fora do código versionado.

2.3 Deploy da API, do banco e do agente

O arquivo versionado src/backend/docker-compose.yml descreve o PostgreSQL 16 e a API, mas o build aponta para a pasta ./api, que não existe, e publica as portas 8000 e 5432. A execução evidenciada não usou esse arquivo. Na pasta src/backend, o comando docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build construiu a imagem asaconnect-api:2.0.0 a partir do Dockerfile da própria pasta. Subiram dois serviços: a API na porta 8080, com o agente do pacote app/agent no mesmo processo, e o postgres:16-alpine no volume backend_postgres_data, sem porta publicada no computador. A rede criada foi backend_default.

O postgres ficou healthy e a API passou a escutar em 0.0.0.0:8080. A conferência da figura foi feita com docker ps, porque os dois contêineres já estavam no ar: asaconnect_api em 0.0.0.0:8080->8080/tcp e asaconnect_postgres somente em 5432/tcp.

Comandos de subida e de conferência, executados em src/backend:

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
docker ps
```

O fluxo da solução é o seguinte. (1) O cliente chama POST /api/v1/auth/login e recebe um JWT. No aplicativo web a base configurada é http://localhost:8080/api/v1. (2) A pergunta segue para POST /api/v1/chat, no campo query. O agente consulta a base institucional e, quando GEMINI_API_KEY está preenchida, aciona o modelo Gemini. (3) A API grava usuários, conversas e auditoria no PostgreSQL pela DATABASE_URL. (4) A API escreve log JSON na saída do contêiner, com método, caminho, status e latência.

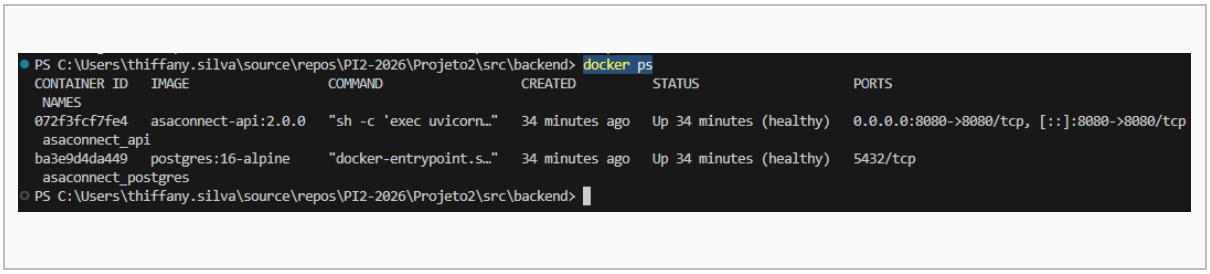


Figura 3 – Stack em execução: API e PostgreSQL, com o banco sem porta publicada no computador.

Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: O comando docker ps foi executado para comprovar que a API e o banco estão ativos, que o agente está embutido na API e que o banco não aparece com porta mapeada para o computador.

2.4 Endereço de acesso controlado

Na execução o endereço da solução é http://localhost:8080. A documentação interativa fica em /docs, a verificação de saúde em /health e a interface web em /app. O GET /health

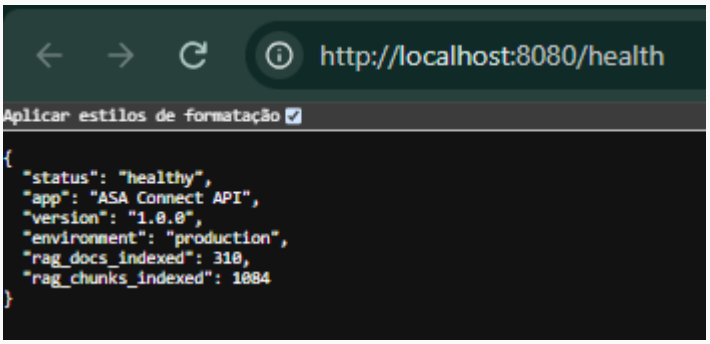
devolve status, app, version, environment, rag_docs_indexed e rag_chunks_indexed. Na captura, o status foi healthy, com 310 documentos e 1884 trechos indexados. A versão definida em app/core/config.py é 1.0.0. Não há endpoint /metrics.

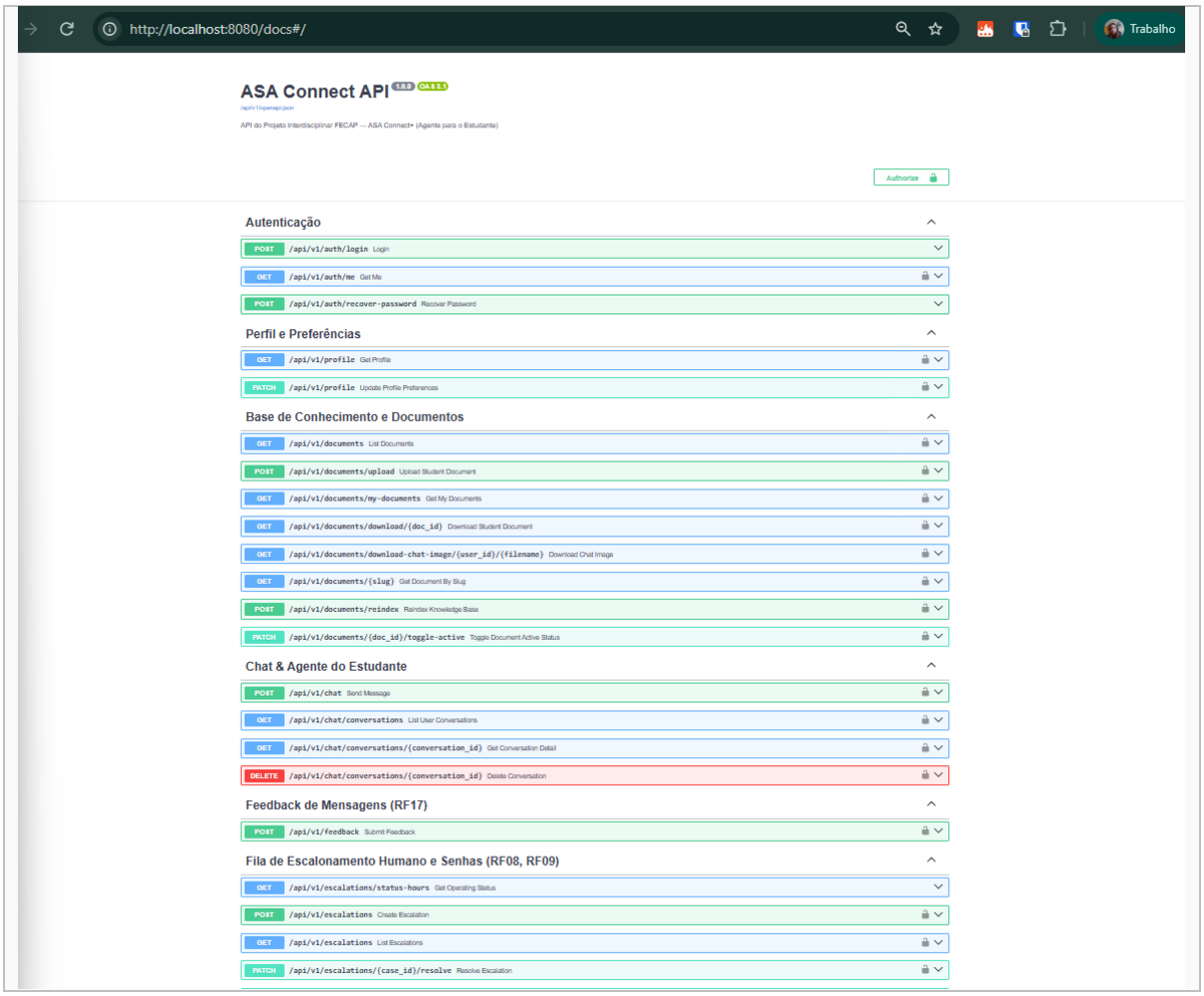
O repositório traz dois descritores de nuvem. Nenhum dos dois foi publicado nesta sessão. src/backend/fly.toml nomeia o aplicativo asa-connect-api, região gru, porta interna 8080, HTTPS, min_machines_running = 1 e 1 GB de memória. A DATABASE_URL desse arquivo é sqlite:////app/asaconnect.db. src/backend/render.yaml descreve o serviço web asaconnect-api em Python, com uvicorn na porta informada pelo provedor e DATABASE_URL em SQLite. O aplicativo web, em src/web/lib/core/constants.dart, aponta para http://localhost:8080/api/v1. O aplicativo móvel, em src/mobile/lib/core/constants.dart, aponta para https://projeto2-7gf7.onrender.com/api/v1. O endereço que respondeu na execução foi http://localhost:8080.

Comando de verificação local:

```
curl http://localhost:8080/health

API: http://localhost:8080/docs
```




http://localhost:8080/docs#/
Trabalho
ASA Connect API
1.0.0
OAS 3.1
API do Projeto Interdisciplinar FECAP — ASA Connect+ (Agente para o Estudante)
Authorize
Autenticação
POST /api/v1/auth/login Login
GET /api/v1/auth/me Get Me
POST /api/v1/auth/recover-password Recover Password
Perfil e Preferências
GET /api/v1/profile Get Profile
PATCH /api/v1/profile Update Profile Preferences
Base de Conhecimento e Documentos
GET /api/v1/documents List Documents
POST /api/v1/documents/upload Upload Student Document
GET /api/v1/documents/my-documents Get My Documents
GET /api/v1/documents/download/{doc_id} Download Student Document
GET /api/v1/documents/download-chat-image/{user_id}/{filename} Download Chat Image
GET /api/v1/documents/{slug} Get Document By Slug
POST /api/v1/documents/reindex Reindex Knowledge Base
PATCH /api/v1/documents/{doc_id}/toggle-active Toggle Document Active Status
Chat & Agente do Estudante
POST /api/v1/chat Send Message
GET /api/v1/chat/conversations List User Conversations
GET /api/v1/chat/conversations/{conversation_id} Get Conversation Detail
DELETE /api/v1/chat/conversations/{conversation_id} Delete Conversation
Feedback de Mensagens (RF17)
POST /api/v1/feedback Submit Feedback
Fila de Escalonamento Humano e Senhas (RF08, RF09)
GET /api/v1/escalations/status-hours Get Operating Status
POST /api/v1/escalations Create Escalation
GET /api/v1/escalations List Escalations
PATCH /api/v1/escalations/{case_id}/resolve Resolve Escalation

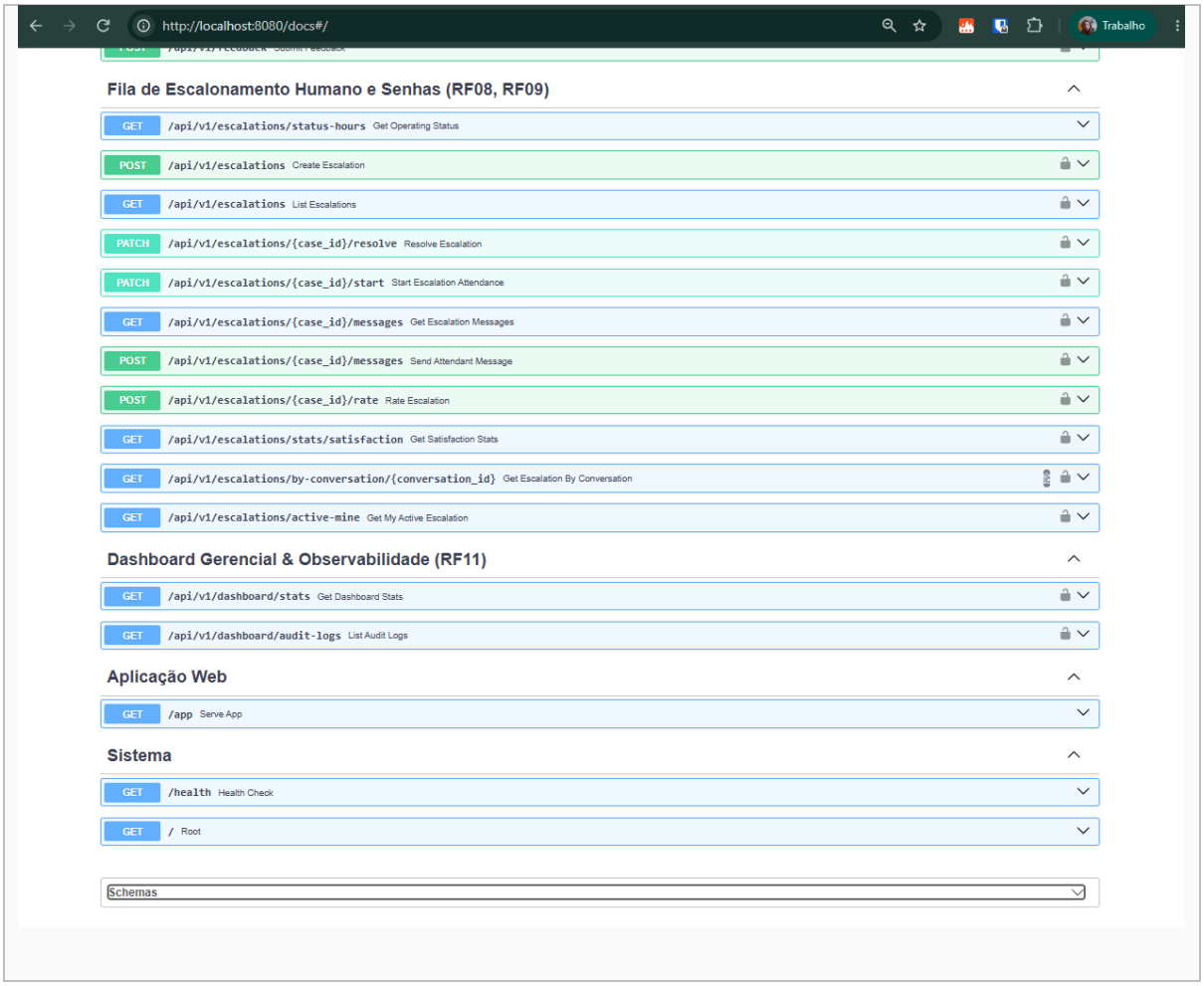


Figura 4 – Resposta de /health e documentação em /docs no endereço controlado da API.
Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: A requisição a /health foi executada para evidenciar o endereço em que a solução responde, com o estado da API e da indexação do agente, sem expor a porta do banco.

2.5 Logs estruturados e métricas de CPU, memória, latência e erros

Os logs saem em JSON pelo módulo app/core/logging.py, na saída padrão do contêiner. Cada registro traz timestamp, level, logger, message, environment, module e line. O middleware de app/main.py acrescenta, nas requisições de negócio, method, path, status_code, duration_ms e client_ip. O campo duration_ms é a latência da requisição. O campo status_code alimenta a leitura de erros. O campo path identifica o endpoint. A consulta é feita com docker logs asaconnect_api.

Não há endpoint /metrics nesta execução, e o requirements.txt não inclui prometheus-fastapi-instrumentator. A latência e o status saem no próprio log JSON, nos campos duration_ms e status_code. CPU e memória dos contêineres são lidas com docker stats --no-stream.

Comandos de coleta:

```
docker logs asaconnect_api --tail 30

docker stats --no-stream
```

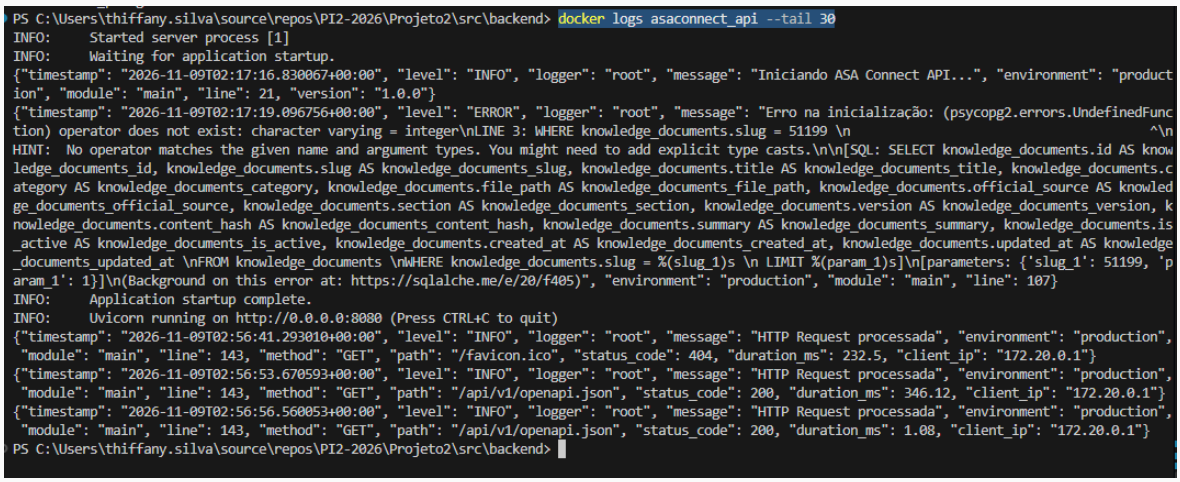


Figura 5 – Log JSON da API com método, caminho, status e latência.
Fonte: elaborado pelos autores (2026).

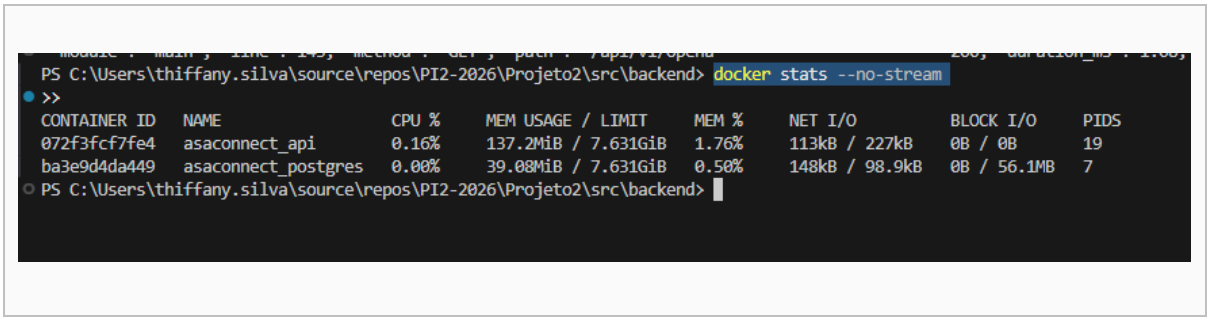


Figura 6 – Consumo de CPU e memória dos contêineres asaconnect_api e asaconnect_postgres.
Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: Os comandos foram executados para reunir o rastro de cada requisição e o consumo de CPU e memória da API e do banco.

2.6 Relatório de monitoramento

O monitoramento desta execução é o conjunto dos logs JSON e do docker stats. Na subida, o log registrou um erro de inicialização: o PostgreSQL recusou a comparação knowledge_documents.slug = 51199, porque slug é texto e o valor enviado foi numérico. A mensagem foi Erro na inicialização. A API mesmo assim concluiu a partida e o Uvicorn passou a escutar em http://0.0.0.0:8080. Depois disso, o GET /health respondeu healthy, com 310 documentos e 1884 trechos indexados. O log também registrou requisições com método, caminho, status e latência. Um exemplo real: GET /api/v1/openapi.json respondeu 200 em 346,12 ms e, na repetição, em 1,08 ms. GET /favicon.ico respondeu 404 em 232,5 ms. O cliente registrado foi 172.20.0.1.

Essa leitura é o relatório de monitoramento da entrega. Ela reúne latência, status HTTP, a falha de indexação e o consumo de CPU e memória dos dois contêineres.

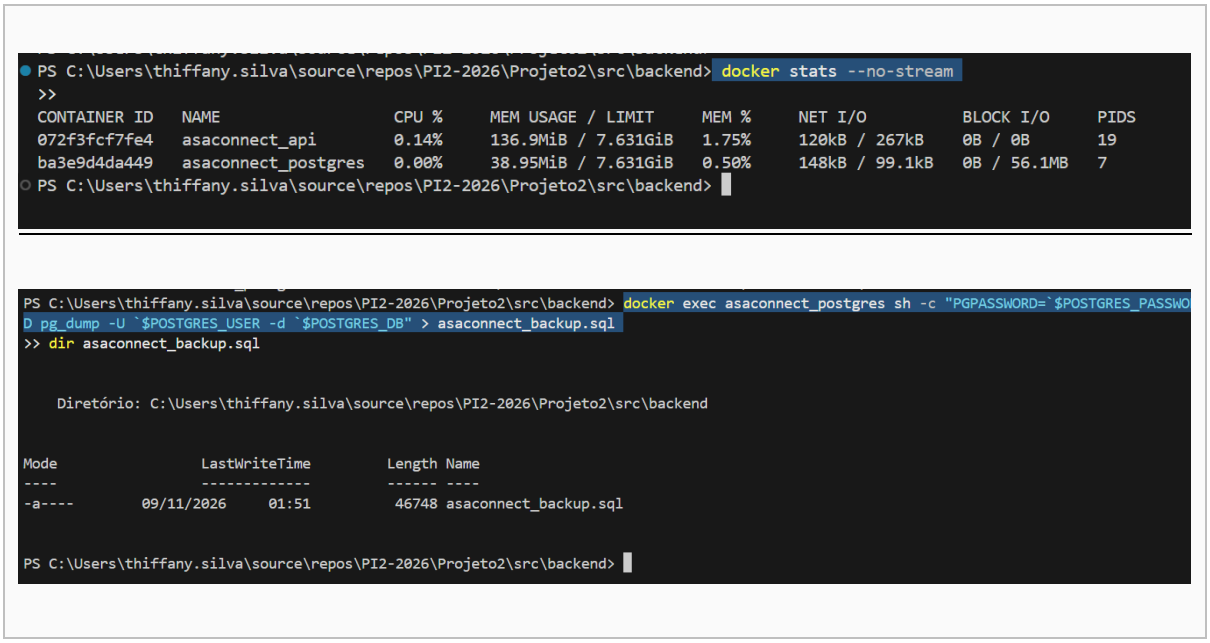


Figura 7 – Log JSON da API com o erro de inicialização do slug e as requisições seguintes.
Fonte: elaborado pelos autores (2026).

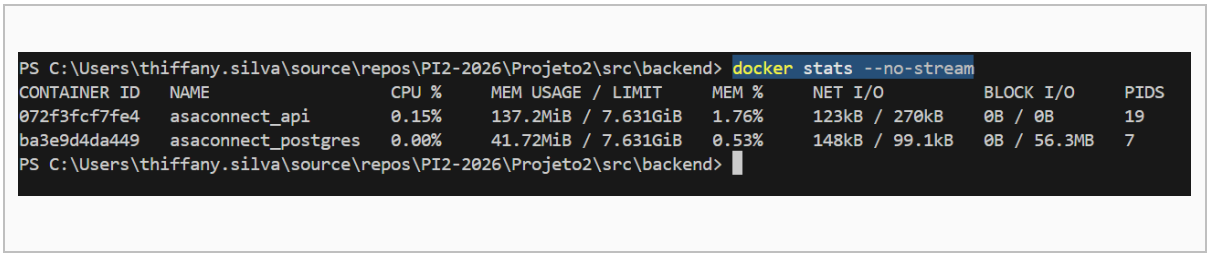


Figura 8 – docker stats dos contêineres asaconnect_api e asaconnect_postgres.
Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: Os logs e o docker stats foram usados como relatório de monitoramento: disponibilidade da API, latência, status HTTP e consumo de CPU e memória.

2.7 Gargalo observado e ajuste indicado

O repositório não traz um ensaio de carga automatizado. A amostra de latência está no log da subida. O gargalo identificado está em app/main.py, na consulta da base: slug é character varying e a consulta comparou esse campo com o inteiro 51199. O PostgreSQL respondeu UndefinedFunction e o log registrou Erro na inicialização. O ajuste é comparar slug como texto, ou usar a chave numérica da tabela. A API HTTP permaneceu no ar. O /health mostrou status healthy, com 310 documentos e 1884 trechos, e o docker ps mostrou os dois contêineres healthy.

A evidência de desempenho é o campo duration_ms do log e o estado healthy dos dois contêineres. Não houve ensaio com usuários simultâneos. Na amostra registrada, GET

/favicon.ico respondeu 404 em 232,5 ms. GET /api/v1/openapi.json respondeu 200 em 346,12 ms e, na repetição imediata, em 1,08 ms. A segunda leitura, bem menor, mostra que o custo alto ficou na primeira montagem da resposta, e não em toda requisição seguinte. O cliente dessas linhas foi 172.20.0.1, o endereço da ponte Docker usada na execução. Essas três medidas são a leitura de latência compatível com o que a API atendeu nesta sessão.

O chat continua sendo o caminho mais caro da API, porque executa o agente sobre a base já indexada. O /health mostrou 310 documentos e 1884 trechos. Nesta sessão não houve tabela de percentis de um teste de carga. O fato medido na subida foi a falha de tipo na consulta do slug, sem derrubar o processo HTTP. No mesmo intervalo, o docker stats mostrou a API em cerca de 0,15% de CPU e 137 MB de memória, e o PostgreSQL em 0,00% de CPU e cerca de 40 MB. Os dois contêineres permaneciam healthy, com a API em 0.0.0.0:8080->8080/tcp e o banco apenas em 5432/tcp. Esse consumo, lido com a API ociosa depois da subida, é a linha de base da entrega. O próximo passo, fora desta sessão, seria repetir a leitura com vários acessos simultâneos ao chat.

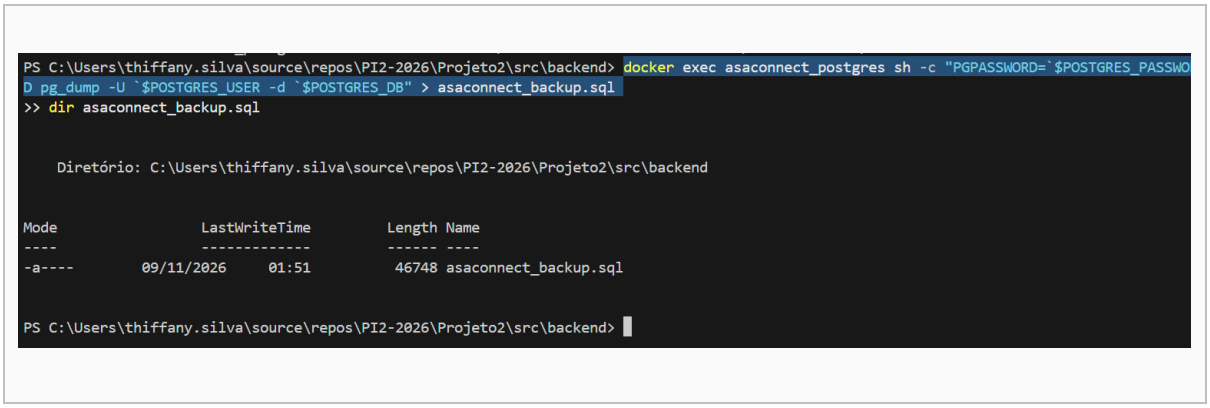


Figura 9 – Trecho do log em que a comparação de slug falha e, em seguida, a API atende HTTP.
Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: O log foi lido para localizar o gargalo da inicialização e para mostrar que a API continuou respondendo depois do erro.

2.8 Backup, restauração, segredos, acesso e recuperação

A autenticação está em app/core/security.py. O token é JWT no algoritmo HS256, com expiração padrão de 1440 minutos. A senha dos usuários da aplicação é armazenada com bcrypt. O controle por perfil usa require_profiles() sobre seis papéis: Aluno, Responsável, Professor, Colaborador, Atendente ASA e Administrador. POST /api/v1/auth/login é público. GET /api/v1/auth/me, GET /api/v1/profile, POST /api/v1/chat e POST /api/v1/documents/upload exigem usuário autenticado. GET /api/v1/dashboard/stats, GET /api/v1/escalations/ e POST /api/v1/documents/reindex aceitam apenas Atendente ASA e Administrador. O login e outras ações sensíveis geram linha em AuditLog.

O .gitignore do projeto bloqueia .env e .env.*, e libera apenas .env.example. A senha do PostgreSQL usada na execução ficou no .env.prod, fora do Git, e não é reproduzida neste

relatório. O contêiner da API roda como user, e na execução o banco não recebeu porta no computador. O docker-compose.yml versionado, ao contrário, publica 5432 e traz uma senha padrão de desenvolvimento. O config.py, o .env.example e o render.yaml também trazem uma chave JWT padrão dentro do repositório. O ajuste é manter a chave de produção só no painel do provedor, ou em arquivo ignorado pelo Git. O fly.toml descreve HTTPS para uma publicação que não foi executada nesta sessão.

O volume backend_postgres_data guarda o banco. A cópia é feita de dentro do contêiner, sem publicar a porta 5432, com pg_dump. A restauração reaplica o arquivo SQL no mesmo banco. A meta é tirar a cópia antes de uma mudança. O RPO é o intervalo entre duas cópias. O RTO é o tempo de recriar os contêineres e reaplicar o arquivo. Esses tempos não foram medidos nesta sessão, e não há scripts/backup.sh nem scripts/restore.sh no código em execução.

Comandos de backup e de restauração:

```
docker exec asaconnect_postgres pg_dump -U asaconnect -d asaconnect_db >
asaconnect_backup.sql
docker exec -i asaconnect_postgres psql -U asaconnect -d asaconnect_db <
asaconnect_backup.sql
```

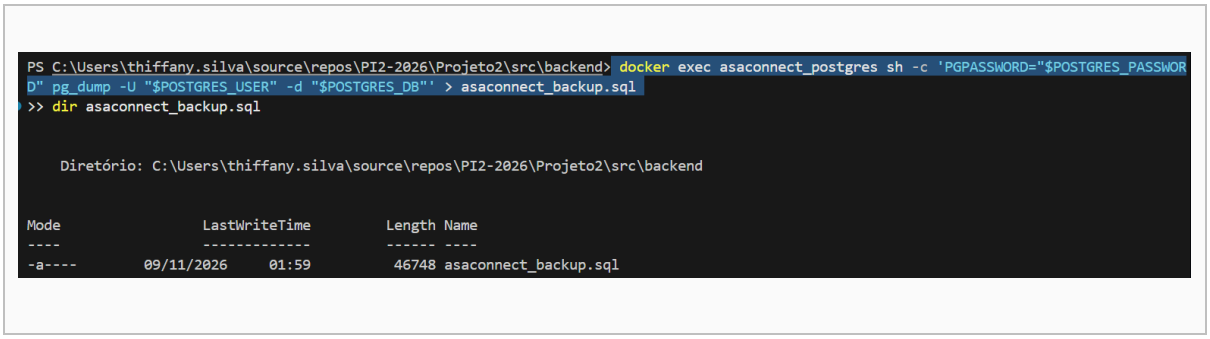


Figura 10 – Backup com pg_dump, a partir do contêiner asaconnect_postgres, sem porta publicada no computador.
Fonte: elaborado pelos autores (2026).

Justificativa da execução no terminal: O comando de backup exporta a base sem publicar a porta do PostgreSQL. A restauração correspondente reaplica o arquivo SQL. Os tempos de recuperação não foram medidos nesta sessão.

2.9 Evidências e repositório

A infraestrutura do projeto está em src/backend: Dockerfile de um estágio, com o usuário user e porta padrão 7860; docker-compose.yml, com PostgreSQL 16; .env.example; fly.toml; render.yaml; app/core/logging.py, app/core/security.py, app/core/config.py e app/main.py. O agente fica em app/agent e a base institucional em knowledge_base. A execução evidenciada construiu, a partir desse Dockerfile, a imagem asaconnect-api:2.0.0 e publicou a API em http://localhost:8080, com o PostgreSQL sem porta no computador. O repositório é https://github.com/2026-2-NCC5/Projeto2.

O endereço de acesso controlado demonstrado é http://localhost:8080, com /health, /docs e /app. O banco dessa execução não possui endereço no computador: no docker ps ele aparece apenas como 5432/tcp. As URLs de nuvem do fly.toml e do aplicativo móvel não foram verificadas nesta sessão.

3 CONCLUSÃO

A Entrega 2 documenta a API, o PostgreSQL e o agente do ASA Connect+ como estão no projeto. A imagem construída do Dockerfile, asaconnect-api:2.0.0, atendeu na porta 8080 com o usuário user. O banco da execução ficou na rede Docker, sem porta no computador, e a senha dessa sessão permaneceu fora do Git. Os logs JSON registram método, caminho, status e latência. O docker stats lê CPU e memória. Na subida, o log registrou o erro de tipo ao comparar o campo slug com o inteiro 51199. A API continuou no ar, e o /health mostrou status healthy, com 310 documentos e 1884 trechos indexados. O ajuste indicado é tratar slug como texto. O backup foi gerado com pg_dump no volume backend_postgres_data. Os descritores fly.toml e render.yaml permanecem no repositório, com SQLite, e não foram a publicação executada. O endereço controlado demonstrado é http://localhost:8080.

As figuras deste relatório são o lugar das capturas da execução. Cada caixa cinza indica o comando correspondente, seguido da justificativa da execução no terminal.